

ICT283_ASSIGNMEN T_2

ICT283: DATA STRUCTURE AND ALGORITHM

1. Design Improvements from assignment 1 feedback:

During the early steps of Assignment 1, my aim was to design the solution in a way that maximize code reuse and future flexibility. I took the project requirement and the importance on modularity and reusable components as a sign that key processes such as unit conversion and statistical analysis should be completely generic.

Based on this, I implemented UnitConvert and Stats as a template class but I later realized that this was not needed as these functions were not showing or having any private member variables to display any behaviour I also realized that every data in this program was represented as float type. All weather records store speed, temperature, and solar radiation as float. The Menu only ever passes Vector<float> to these functions. This means that although the template was flexible, the system had no situation in which other types would ever be used.

- The previous Stats<T> class contained only stateless methods that did not depend on any internal data members. Since there is no internal state to protect, a class would not add any abstraction value. Even though the original class was declared as template<typename T>, every method accepted const Vector<float>& and so all statistical methods were changed into non template free functions inside a dedicated header Stats.h and implementation file Stats.cpp. this design now displays the actual usage in the program only calculates statistics for floating-point weather data. This also improves memory allocation as only one single copy of each methods is created. In menu now (Stats<float>::calculateMean() → calculateMean())
- The UnitConvert<T> class did not maintain any private state it simply applied mathematical conversions to vectors. I realized that capturing such stateless behavior in a class violates the design intent of class: entity with state and behavior. These methods were changed into free functions within UnitConvert.h and UnitConvert.cpp. This keeps the design minimal, allows direct reuse without object construction in menu
- The previous design implemented FileReader as a class containing one public method This structure was based on the assumption that reading file paths should be encapsulated inside an object. However, after reviewing the lecturer's feedback and reconsidering the purpose of this component, it became clear that FileReader did not satisfy the criteria for a class it had no private data members, no internal state. It did not store a file path. The class existed solely to group together a file-loading function. Since FileReader had behaviour but no state, the class abstraction added no real value which is why The new design replaces FileReader with a set of free functions, where the helper function is declared as a private static function inside the .cpp file. This improves cohesion (functions do one thing), avoids needless construction, and makes things simpler. This also treats file reading as a clean facility following the principle that utility operations belong to functions, not classes
- The previous RecordParse class encapsulated a single parseLine() function with no internal state. It was then converted into a free function ReadFile() under RecordParse.h and RecordParse.cpp. The parser now behaves as a

pure function, returning a WeatherRecord from an input string without needing an object instance.

The previous design implemented RecordParse as a class, even though it did not represent any real object in the system. The class contained no member variables. Its only duty was to provide a ReadFile() function that parsed a CSV file into WeatherRecord objects. RecordParse provided behaviour only, and that behaviour depended on the input parameters passed to the method. This made the class abstraction unnecessary

In main.cpp the method was always called in this way:

```
RecordParse reader;  
reader.ReadFile(filename);
```

which indicated that RecordParse was just a namespace, not an object with relevant members. The changed solution replaces the class with a free function:

```
Vector<WeatherRecord> ReadFile(const std::string& filename);
```

This design reduces unnecessary coupling, simplifies main.cpp, and supports exactly with the lecturer's importance on using classes only when they encapsulate meaningful state.

- Previously, my Date class included iostream such as printing which violates the Single Responsibility Principle. After reviewing the feedback, I changed Date into a pure data model.
- Extended Rationale for Making WeatherRecord a Class:
 - ⊗ WeatherRecord is implemented as a class because it represents a complete weather record that has a strong internal state consisting of date, time, wind speed, temperature, and solar radiation. These pieces of data belong together and must always change as a component. Using a class imposes this cohesion. The class captures this state using private data members, ensuring that external code cannot modify weather values without using the provided interface. This serves as a protection imposing data integrity; for example, wind speed cannot be replaced by an invalid value unless explicitly set through the constructor. A class is also suitable because WeatherRecord is not just an inactive structure; it participates in higher-level behaviours such as being stored inside the BST, Map, and aggregator. The class abstraction allows the rest of the program to treat a weather record as a self-contained object with expected behaviour, rather than independent variables that could easily get out of sync. By encapsulating the weather record into a class, the program can be extended later (e.g. validation rules or comparisons) without breaking the rest of the code. This would not be possible if WeatherRecord were a set of free variables or a struct with public fields. So, the choice to implement WeatherRecord as a class offers cohesion, data integrity, future extensibility.

1. Data Dictionary for Bst,Map, Vector:

Bst<T>				
Name	Type	Protection	Description	Rationale
F1Type	typedef void (*f1Type)(const T&)	Public	Function pointer type representing the action callback applied to each node's data during traversal.	Enables external functions to process or collect node data dynamically without coupling the BST to application-specific logic.

f2Type	typedef bool (*f2Type)(const T&)	Public	Function pointer type representing the condition check for traversal. Determines whether a node's data should be processed by the callback.	Allows selective traversal by filtering nodes that meet certain conditions (e.g., month filter in weather data).
Bst()	Constructor	Public	Default constructor that initializes an empty BST with root set to nullptr.	Ensures safe, initialization of an empty tree.
~Bst()	Destructor	Public	Recursively deletes all nodes in the BST using deleteTreeRecursive().	Prevents memory leaks by deallocating all dynamic nodes.
Bst(const Bst& other)	Constructor	Public	Copy constructor that creates a deep copy of another BST using copyConstRecursive().	Ensures proper deep copy to prevent shared memory of nodes.
operator=(const Bst& other)		Public	Assignment operator deletes the current tree and deep-copies another BST.	Provides correct assignment while avoiding memory leaks
insert(const T& value)	void	Public	Inserts a new node containing value into the BST while maintaining sorted order.	insertion for building the tree; used by WindlogAggregator to store Date objects.
search(const T& value) const	bool	Public	Searches the BST for a given value. Returns	Provides efficient search (average $O(\log$

			true if found, otherwise false.	n)) to check element found.
inOrderTraversal (f1Type callback, f2Type condition) const	void	Public	Performs an in- order (Left– Root–Right) traversal of the BST. Applies the callback function to each node’s data only if the condition function returns true.	Provides sorted traversal of BST data, while allowing conditional filtering for specific operations (e.g., selecting dates by month).
preOrderTravers al(f1Type callback, f2Type condition) const	void	Public	Performs a pre- order (Root– Left–Right) traversal. Executes the callback on each node’s data if the condition is met.	Supports pre- order processing where parent nodes are visited before children useful for cloning
postOrderTraver sal(f1Type callback, f2Type condition) const	void	Public	Performs a post- order (Left– Right–Root) traversal. Applies the callback function only to nodes that satisfy the condition.	Used for operations where children must be processed before parents, such as recursive deletion
Node	struct	Private	Represents a single node in the BST containing the stored data and child pointers.	Encapsulates tree elements and maintains the hierarchical structure.
Node::data	T	Public (within struct)	The data value stored in this node.	Allows generic data
Node::left	Node*	Public (within struct)	Pointer to the left child node.	Represents values smaller

				than the current node's data.
Node::right	Node*	Public (within struct)	Pointer to the right child node.	Represents values greater than the current node's data.
Node(const T& value)	Constructor	Public (within struct)	Initializes a new node with data = value and left and right pointers set to nullptr.	Ensures safe initialization of each node.
root	Node*	Private	Pointer to the root node of the BST.	Serves as the tree's entry point; private for data protection and encapsulation.
insertRecursive(Node* node, const T& value)	Node*	Private	Recursively inserts a new value in the correct order. Returns a pointer to the updated subtree root.	Simplifies insertion logic and maintains BST ordering property.
searchRecursive(Node* node, const T& value) const	bool	Private	Recursively searches for a value in the BST starting from root.	recursive lookup separated from the public interface.
deleteTreeRecursive(Node* node)	void	Private	Recursively deletes all nodes in the subtree rooted at node.	Used by the destructor and assignment operator to prevent memory leaks.
inOrderRecursive(Node node, f1Type callback, f2Type condition) const*	void	Private	Helper method that recursively performs in-order traversal starting from a node. Applies	Implements the ordered traversal logic used by inOrderTraversal() while

			the callback to each node's data that satisfies the condition.	preserving encapsulation.
preOrderRecursive(Node node, f1Type callback, f2Type condition) const*	void	Private	Helper method that recursively performs pre-order traversal starting from a node.	Supports internal logic for preOrderTraversal(), allowing pre-processing of nodes before their children.
postOrderRecursive(Node node, f1Type callback, f2Type condition) const*	void	Private	Helper method that recursively performs post-order traversal starting from a node. Applies the callback to nodes that satisfy the condition after visiting both subtrees.	Supports postOrderTraversal() functionality for safe cleanup
copyConstRecursive(Node* node)	Node*	Private	Recursively creates a deep copy of the given subtree and returns the new root node.	Ensures safe cloning behavior for both the copy constructor and assignment operator.

Map<K,V>				
Name	Type	Protection	Description	Rationale
Map()	Constructor	Public	Default constructor that initializes an empty internal std::map.	Ensures the container starts empty

insert(const K& key, const V& value)	bool	Public	Inserts a key–value pair into the map. Returns true if insertion succeeds, false if the key already there.	Provides safe insertion while preventing duplicate keys
contains(const K& key) const	bool	Public	Checks whether a given key exists in the map. Returns true if found, false otherwise.	Enables safe key existence checks before inserting data; used by WindlogAggregator.
get(const K& key)	V&	Public	Returns a non-const reference to the value associated with the key. Throws an exception if the key does not exist.	Allows modification of existing entries; supports data aggregation method.
get(const K& key) const	const V&	Public	Returns a const reference to the value associated with the key. Throws an exception if the key does not exist.	Provides read-only access to mapped values for reporting and statistics.
data	std::map<K, V>	Private	The internal container that holds key–value pairs.	Encapsulates the STL implementation to hide complexity and maintain abstraction.

Vector<T>				
Name	Type	Protection	Description	Rationale
m_vector	std::vector<T>	Private	Internal STL container used to store elements dynamically.	Using std::vector internally provides automatic memory management, resizing, and efficiency while still allowing us to encapsulate it for control and consistent interface with custom methods.
Vector()	Constructor	Public	Default constructor that initializes the internal vector.	Keeps interface simple while allowing future extension (e.g., custom capacity). Demonstrates correct use of template constructors.
operator[](int index)	T&	Public	Provides read-write access to an element at the given index.	Allows array-like access syntax (myVector[i]) for modification.
operator[](int index) const	const T&	Public	Provides read-only access to an element at the given index.	Const overload enforces fixity for const objects and prevents accidental modification.
size()	int	Public	Returns the current number	Enables checking vector length before

			of elements in the vector.	indexing, preventing out-of-bounds access.
pushBack(const T& element)	void	Public	Appends a new element to the end of the vector, automatically expanding capacity if needed.	Simplifies adding new data dynamically and matches common STL vector behavior, providing familiarity.
isEmpty()	bool	Public	Returns true if the vector contains no elements.	Provides a quick way to check if the vector has data before performing operations, improving code safety.

Design Rationale Summary

1. Binary Search Tree:

- Design choices
- The Bst<T> class provides a minimal and complete implementation of a generic binary search tree used by the WindlogAggregator to store and order Date object in serial order. It supports insertion, searching, and three traversal modes (in-order, pre-order, post-order), enabling data to be processed in different sequences.
- Traversal operations now use two function pointers instead of one:
 - f1Type (void (*)(const T&)) a callback that defines the action to be performed on each visited node (e.g., printing, collecting, or summing values).

- f2Type (bool (*)(const T&)) a condition function that determines whether the callback should execute for a particular node.
- The new conditional parameter design achieves the same flexibility as a void* context but with stronger type safety and encapsulation. External classes (e.g., WindlogAggregator) can now perform context-dependent traversals by defining simple Boolean condition functions (e.g., “Is this date in the target month?”), while the BST itself remains unaware to what the condition means.
- Implements the Rule of Three: destructor, copy constructor, and assignment operator—to safely manage dynamically allocated nodes and prevent memory leaks.
- Recursive helper functions make clean, readable logic for insertion, search, and traversal operations.
- Each traversal helper (inOrderRecursive, preOrderRecursive, postOrderRecursive) now takes both a callback and a condition function pointer. This ensures that the traversal logic remains generic while granting the caller complete control over which nodes are processed and how.
- The private Node struct is there only to offer the internal structure of the BST. By nesting it within the class and make it private, I prevent client code from by chance corrupting node links. This design attains strong encapsulation if the internal structure ever changes (e.g., to AVL), external code remains unchanged.

- **BST In-Order Traversal Implementation Rationale**

- Design Approach for getAllRecordsForMonth()
 - Problem: The requirement to retrieve all weather records for a specific month across multiple years presented a data association problem. The BST's in-order traversal ability provided a solution for accessing dates in chronological order.
 - During traversal the condition (bool (*)(const Date&)) evaluates whether a node matches a user-stated condition e.g., whether the Date belongs to the target month. If condition(node->data) returns true, the callback (void (*)(const Date&)) executes, performing an action such as pushing the date into a Vector<Date>.

```
s_targetMonth = month;
```

```
s_collected = &collectedDates;
```

```
m_dateTree.inOrderTraversal(collectMonthDates, isTargetMonth);
```

- Here, isTargetMonth serves as the condition and collectMonthDates as the callback. This implementation keeps the BST's traversal logic generic and application-independent while enabling selective traversal within WindlogAggregator.

- Benefits of this: eliminates unsafe void* context casting, Improves type safety, BST handles structure and traversal. WindlogAggregator defines the operational logic.
- In my `getAllRecordsForMonth()` implementation in `windlogaggregator` class, it specifically helps month-based filtering during chronological traversal. This approach helped me eliminate global variables from my code

Pros	Cons
Provides $O(\log n)$ search and insertion making it ideal for sequential access to weather records.	As tree is unbalanced, so insertion of sorted data could reduce performance to $O(n)$.
Keeps sorted order of Date keys, shortening in-order traversal for sequential reporting. Eliminates the need for sorting	BSTs do not handle duplicates by itself while my program permits equal Date values, duplicate records could lead to redundant nodes.
Uses recursive traversal and function pointers for traversal demonstrating flexible tree traversal. Supports open/closed principle	Recursion and node management rise implementation complexity compared to iterative-based approach.
pointer-based design lets tree to grow as needed without pre-allocation.	

Reason for choosing a recursive approach

- A recursive approach was chosen over an iterative one to highlight theoretical clarity and ease of code.
- Recursive functions express hierarchical relation, where each node calls the same operation on its left and right subtrees, matching the tree's logical structure. This improves readability and maintainability
- Since the dataset size in this project is restrained and does not risk deep recursion, the recursive approach remains efficient and safe.
- An iterative implementation would require explicit use of stacks, which adds code complication without providing performance benefits for the current dataset size.

Alternate considerations:

- A self-balancing tree (e.g., AVL) could avoid bad performance on sorted datasets.
- An iterative bst implementation might have lessened recursion depth but make codes design harder to read

- A hash table (future improvement) could provide $O(1)$ average lookup for unordered searches but would lose sorted traversal capability.

This final BST design uses dual function pointers for callback and condition, achieving both flexibility and type safety.

It preserves the BST's generic, reusable interface while allowing selective traversal in client code such as WindlogAggregator.

The `inOrderTraversal()` method demonstrates this design by enabling chronological, condition-based traversal of weather data without introducing global variables, external state, or unsafe casting.

2. Map:

- The `Map<K, V>` class is a minimal wrapper around the C++ Standard Library's `std::map`.
- The internal `std::map` (data) is private, which hides the STL complexity from client code, ensuring encapsulation and abstraction
- Uses templates (`<K, V>`) to handle different key and value types.
- It provides only the method needed for the program: `insert()`, `contains()`, and `get()`, following the minimal and complete class principle.
- Both `const` and `non-const` `get()` methods maintain `const`-correctness
- The design supports all aggregation methods required in `WindlogAggregator` while preventing exposure of STL iterators or direct access to internal data.
- This abstraction keeps client code easy to use, reduces dependencies on STL, and ensures that if the container were ever changed (e.g., to a vector or hash table), no client-side code would need update.
- The `Map<K, V>` template class provides a minimal and complete key-value association structure, used widely within the program to assist domain-

specific data objects such as `MonthYearKey` → `MonthlyData` and `Date` → `Vector<WeatherRecord>`.

It serves a similar role to `std::map`, but with a simplified, assignment-compliant interface that explicitly separates insertion, retrieval, and existence checking.

- The class exposes only the fundamental methods `inserts()`, `contains()`, and `get()` which together provide all the essential operations required for associative storage
- Each key–value pair is stored in sorted order (internally implemented using a Bst), ensuring that lookups and insertions operate in $O(\log n)$ time on average.

This guarantees likely performance even when processing large weather datasets across multiple files.

- By avoiding convenience overloads such as `operator[]`, the design remains minimal, safe, and explicit preventing unintended side effects such as implicit key insertion or default construction of missing entries.
- **Where the Map is Used in the Program**

1. In `WindlogAggregator`

The Map template is instantiated as:

```
Map<MonthYearKey, MonthlyData> m_monthlyMap;
```

This map serves as the **core monthly aggregation store**, where:

- **Key:** `MonthYearKey` (combination of year and month)
- **Value:** `MonthlyData` (object storing vectors of wind speed, temperature, and solar radiation)
- It enables fast access to all weather data grouped by month and year.

For instance, when processing a new record:

```
MonthYearKey key(year, month);
```

```
if (!m_monthlyMap.contains(key)) {  
    MonthlyData newData(year, month);  
    m_monthlyMap.inserts(key, newData);  
}
```

```
MonthlyData& monthlyData = m_monthlyMap.get(key);  
monthlyData.addRecord(record);
```

- This ensures that all records belonging to the same (year, month) pair are stored and aggregated together.
- When the menu system later requests monthly statistics (e.g., average wind speed or temperature), the aggregator efficiently retrieves the relevant `MonthlyData` object. This design avoids repeated scanning of all records and supports **constant-time access** to already-grouped data once keys are created.

2. In `DateRecMap`:

- The Map is also instantiated as:

- **Map<Date, Vector<WeatherRecord>> m_dateToRecords;**

- Here, it functions as a **daily record index**, mapping each unique Date to a (Vector) of WeatherRecord objects. This provides efficient retrieval of all records corresponding to a specific day.
- For example:

```
if (!m_dateToRecords.contains(date)) {
    Vector<WeatherRecord> newRecordList;
    m_dateToRecords.inserts(date, newRecordList);
}
m_dateToRecords.get(date).pushBack(record);
```
- ensures that multiple readings from the same day are grouped together automatically. This is crucial when exporting data or calculating day-based averages and correlations.
- When the BST traversal in WindlogAggregator::getAllRecordsForMonth() collects all dates for a specific month, those dates are passed to this map to quickly retrieve all corresponding records for that date.

Pros	Cons
Fast key lookup: Average $O(\log n)$ access using <code>std::map</code>	Memory overhead higher due to tree node management within <code>std::map</code> .
Automatic ordering: Keeps data sorted by key (year/month), making sequential output simpler.	Custom wrapper offers only basic methods (insert, contains, get), limiting other STL features like iterators
Prevents multiple entries for the same month or date, ensuring data consistency.	Insertion cost slower than hash-based substitutes for very large datasets.
Hides STL implementation details, lead to a clear abstraction layer	

Alternative consideration:

- A `std::unordered_map` could improve lookup time to average $O(1)$ but would lose key ordering, making output and monthly summaries less suitable
- As part of my design examination, I also applied a Map using my own Vector class instead of `std::map`. This stored key–value pairs in a dynamic array and used simple for loops to search for keys. While this approach has **linear time complexity** for searches and insertions because the vector must be examined element-by-element ($O(n)$ time per lookup). By difference, the `std::map` wrapper used holds logarithmic search and insertion complexity ($O(\log n)$). Given that the dataset

could increase in size beyond the present records used, the performance of `std::map` is a better fit, so I opted to keep the wrapped `std::map` implementation while seeing the vector approach as an alternative.

- By encapsulating the map behind methods like `inserts`, `contains` and `get`, this dissociate the rest of my code from the choice of container. If a future must use an `unordered_map` ($O(1)$ average-case lookups) I can change the original container inside the wrapper and leave the public interface unmoved. In short, an encapsulated map provides the effectiveness of associative storage while giving you control over how it is used and letting for clean, maintainable progress of the code.
- For this project, `std::map` offers the best trade-off between simplicity, and reliable ordering.

3. Vector:

- Used extensively for storing variable-length collections such as wind speeds, temperatures, solar radiation readings, and parsed records in `RecordParse` and `MonthlyData`.
- In assignment 1, I used my own `Vector` class to establish understanding of dynamic arrays and memory management. But, after being allowed to make use of `std::vector` in Assignment 2, I replaced my custom `Vector` with `std::vector`. This choice was made even after admitting some known disadvantages of STL vectors.
- I replaced my custom `Vector` with `std::vector` because the STL version is suggestively safer, more efficient. Although `std::vector` can over-allocate memory and may momentarily waste space, these disadvantages are negligible compared to the benefits of automatic memory management and reduced code complexity. Using `std::vector` removes the need to implement my own memory methods (Rule of Three) in every class that uses it as an instance, this decreases coupling across classes, and allows the design to focus on weather analysis rather than container application.

Pros	Cons
Automatically grows in size to add new elements.	Inserting or deleting elements (except at the end) is $O(n)$.
constant time element retrieval by operator[].	A custom implementation is more fault-prone than STL <code>std::vector</code> .

Vector was chosen over arrays and linked lists because arrays were rejected due to static allocation whereas linked lists provide dynamic size but slower random access ($O(n)$)

Rationale for In-Order Traversal in WindlogAggregator

- The WindlogAggregator class uses the Binary Search Tree ($Bst<Date>$) primarily to maintain all unique dates from the dataset in chronological order and to enable efficient month-based filtering without repeated sorting or linear scanning.
`m_dateTree.inOrderTraversal(collectMonthDates, isTargetMonth);`
- demonstrates the assignment's key requirement of using function pointers to drive tree traversal logic
- **Design:** my objective was to retrieve all Date objects belonging to a user-specified month across multiple years, while preserving chronological order.
- **Problem:** The BST itself must remain generic and unaware of domain logic (e.g., months, years, or weather data).
- **Solution:** separate traversal mechanics (inside $Bst<T>$) from contextual behavior (inside WindlogAggregator) via two function pointer parameters

Parameter	Type	Role in Traversal	Defined In	Description
collectMonthDates	<code>void (*)(const Date&)</code>	Callback (f1Type)	WindlogAggregator	Executed whenever a node meets the condition. Adds that Date to a dynamically growing <code>Vector<Date></code> (<code>s_collected</code>).
isTargetMonth	<code>bool (*)(const Date&)</code>	Condition (f2Type)	WindlogAggregator	Determines whether the current node's Date should be processed by the callback. Returns true when <code>Date.GetMonth()</code> matches <code>s_targetMonth</code> .

These two parameters are passed to the `inOrderTraversal(f1Type callback, f2Type condition)` of the `Bst<Date>` class, which performs a recursive Left-Root-Right visit. At each node:

1. The condition function is evaluated:

```
if (!condition || condition(node->data))
```

```
    callback(node->data);
```

2. If true, the callback executes, collecting the matching `Date` into a local `Vector<Date>`.
3. The in-order algorithm visits nodes in ascending order (by date), ensuring that `collectedDates` is already sorted. The traversal in `Bst` remains unaware of how `Date` is interpreted month filtering logic is completely defined through the function pointers in the aggregator. No global variables or I/O are used inside the `Bst` static members `s_targetMonth` and `s_collected` act as controlled links only within `WindlogAggregator`, keeping the separation of data structure and application logic.
4. Operational Flow: The user selects a month in the menu (e.g., January = 1). `s_targetMonth` is set to that value, and `s_collected` points to a local `Vector<Date>`. The BST traversal is invoked:

```
m_dateTree.inOrderTraversal(collectMonthDates, isTargetMonth);
```

Each matching date is pushed into `collectedDates`. After traversal, static pointers are reset (`nullptr`), and the collected dates are used to fetch daily records from `m_dateRecordMap`

Rationale for Statistical Methods: Pearson Correlation (sPCC) and Mean Absolute Deviation (MAD)

The Stats file was extended with two new statistical methods `calculateSpcc()` and `mad()` to satisfy the Assignment 2 requirements for correlation and dispersion analysis of weather dat.

- a. **Sample Pearson Correlation Coefficient (sPCC):** The method `calculateSpcc(const Vector<float>& x, const Vector<float>& y)` calculates the Sample Pearson Correlation Coefficient, measuring the direction of the linear relationship between two datasets (e.g., wind speed vs temperature, wind speed vs solar radiation, temperature vs solar radiation).

Design approach: It begins by verifying input validity ($n < 2$ or unequal vector sizes $\rightarrow$ returns 0.0) to ensure statistical security. The mean of each vector is calculated using `calculateMean()`. The method then calculates three core summations and then the final correlation coefficient following the standard statistical definition and avoids integer truncation by using double. To ensure correctness, results from `calculateSpcc()` were compared against Microsoft Excel's built-in `=PEARSON()` function, as recommended in the Assignment 2 QandA. Tests using January weather data produced matching results to within rounding error (difference < 0.001), confirming the reliability of the implementation. This design performs $O(n)$ time with a single traversal loop. Returns 0.0 safely for insufficient data, preventing division-by-zero errors.

- b. **Mean Absolute Deviation (MAD)**: The method `mad(const Vector<float>& data)` calculates the Mean Absolute Deviation, representing the average degree of deviations from the dataset mean

Design approach: Chosen to complement the standard deviation in output Option 4 ("Average wind speed, temperature, and total solar radiation with (stdev, mad) format"). Implemented using a simple linear pass: compute mean $\rightarrow$ gather absolute deviations $\rightarrow$ divide by n . This avoids squaring deviations, providing a more essential measure of variability that is less sensitive to outliers.

Use: used in the `WindlogAggregator` during CSV export for each month's statistics. The resulting output format follows the specification where values in parentheses are (stdev, mad).

Rationale for MonthYearKey, DateRecMap, DateUtils:

- The **DateRecMap** class serves as a wrapper around a **Map<Date, Vector<WeatherRecord>>**. It encapsulates the relationship between dates and their matching weather records but does not perform domain-level logic. While this design helps modularity and hides data-structure details from higher-level components (such as `WindlogAggregator`), it could be simplified to a type alias if no added behaviour is required. Future additions could include methods for date-range retrieval to improve its behaviour.

- The **getMonthName()** method is declared as a free utility function within DateUtils namespace because it is used by the Menu class to display human-readable month names. This improves modularity and adheres to the Single Responsibility Principle by separating date-related formatting utilities from data aggregation logic in WindlogAggregator. It promotes reusability across modules (e.g., Menu, MonthlyData) without class coupling. Implementing this function as a free function rather than a member method ensures that formatting responsibilities are not mixed into the data-holding Date class, preserving its focus on representing a valid date.
- The **MonthYearKey** was refactored from a class into a simple struct since it only stores two integers and offers free function of ordering and equality operators. This change improves simplicity, readability, and performance while preserving all behaviour for use as a map key. It also adheres to the principle of minimal abstraction using a lightweight value type where no encapsulation or complex behaviour is required.
- The **DateUtils** module was implemented as a group of free utility functions that operate on Date and MyTime objects without changing their internal implementation.
 - This design follows the Single Responsibility Principle (SRP) by isolating generic date and time-related operations such as comparison, output formatting, and month-name conversion from the core data classes (Date and MyTime).
 - Comparison operators (<, ==, !=, <=, >=, >) were defined as non-member friend-style free functions. This supports symmetric type conversions and allows both operands to be treated equally in comparisons, following to C++ best practice for operator design. It also promotes code reusability, as these operators can be used in algorithms without modifying the Date class.
 - Similarly, the stream output operator (operator<<), allowing Date objects to be printed directly using std::cout in a readable format. This supports in-built I/O usage while maintaining encapsulation no internal data members of Date are directly exposed
 - By grouping these free functions into a utility module, the program attains high cohesion (all date/time-related helpers in one place) and low coupling (core classes remain independent of output or comparison logic). This approach shortens coming methods for example, adding formatting options, or custom comparison rules can be done without changing the Date or MyTime classes.

Bst and Map:

Both the Binary Search Tree (BST) and Map structures show essential but distinct roles in the weather data analysis.

The BST (Bst<Date>) is responsible for storing and ordering unique Date objects in chronological sequence, providing a basis for time-based traversal and ordered access.

In contrast, the Map (Map<Key, Value>) extends this concept into a key–value pair, allowing grouped access to datasets such as

MonthYearKey → MonthlyData and Date → Vector<WeatherRecord>.

Theoretically, the Map can be viewed as a BST-created container that builds on the same recursive ordering logic but introduces an additional layer of abstraction for paired data retrieval.

From a computational view, both structures share similar performance: insertions, searches, and lookups all operate in $O(\log n)$ time on average, with a potential $O(n)$ degradation only in the rare case of a completely unbalanced tree. However, the BST is improved for ordered traversal and sorting operations (for example, the in-order traversal used in `getAllRecordsForMonth()`), while the Map excels at fast direct lookups by key without requiring full traversal. The BST was ideal for maintaining a chronological structure of all dates, while the Map was well suitable for grouped aggregation, where data are accessed repeatedly through discrete keys (month–year pairs).

Given the large size of the dataset, the Map finally proved to be more efficient and practical abstraction for data aggregation and retrieval tasks. Its explicit key-based access avoided repeated traversals through the full BST and allowed quick lookups of monthly data using composite keys.

The BST remained valued where ordering and traversal were essential such as making sorted outputs but the Map offers better scalability and cleaner client-side integration for bulk data.

Together, the two structures complemented each other: the BST ensured order integrity, while the Map provided efficient indexing and retrieval for big size weather records.

Flow of program:

In my program, each data structure plays a role that matches its strengths

- I use (**BST**) to store and **arrange Date objects in sorted order**. This lets efficient chronological traversal of weather data when I need to regain records for specific months. The recursive traversal with function pointers makes the BST reusable for various methods without rewriting the traversal logic.
- I use a **Map** (key by MonthYearKey) for grouping and **fast access of MonthlyData objects**. This structure **allows me to locate all data belonging to a specific month and year**, which is required for computing averages and

standard deviations. The Map allows constant-time lookups without linear search through all records.

- I use a Vector as my dynamic array for storing objects like wind speeds, temperatures, and solar radiation readings per month. It equips fast access and auto-resizing behavior.

Together, these structures work: the BST organizes by date, the Map organizes by month and year, and Vectors store the raw numerical data.

- **Option 1** (Average wind speed and standard deviation for a month and year): uses the **Map** to quickly locate the MonthlyData object fitting to the entered year and month. Inside that object, the real wind speed values are stored in a **Vector**, which is then passed to the stats functions to compute the mean and standard deviation. Here, the Map supplies the lookup, and the Vector delivers the data.
- **Option 2** (Average ambient temperature for each month of a year): relies on the **Map** for month-to-month access. For every month, the program restores the stored **Vector** of temperature readings from MonthlyData.
- **Option 3** (Sample Pearson Correlation Coefficient for a month across all years): This is where the **BST** is used mostly. The program traverses the BST (which stores all unique dates in chronological order) utilizing recursive traversal with a function pointer approach. As it visits each date, it gathers all affiliated WeatherRecords for the desired month from the **DateRecordMap**. These records are then stored in **Vectors** for wind speed, temperature, and solar radiation to execute correlation computations. So, BST runs traversal, Map supplies record grouping, and Vectors hold numerical values for computation.
- **Option 4** (Export comprehensive data to CSV): utilizes the Map to loop through each month of a chosen year, retrieves relevant **Vectors** of wind, temperature, and solar data, computes the statistics, and writes them to a file. The BST isn't needed here because the **Map** already arranges everything by month.

Limitations identified/future improvement:

- Although my current design uses a Binary Search Tree (BST) and a Map structure to organize weather data, future versions could use hash tables to improve performance
- The BST and Map implementations both provide $O(\log n)$ average-case time complexity for insertion, lookup, which is good for moderate data sizes and offers the additional benefit of maintaining sorted order.
- However, as the number of weather records is huge a hash-based structure could reduce many of these operations to $O(1)$ average time by computing a hash value for each key (such as Date or MonthYearKey).
- For example, checking for duplicate dates or retrieving all weather records for a specific date currently requires logarithmic-time tree lookups. A hash map could've been used to store unique Date keys, enabling constant-time

duplicate detection and faster date-to-record mappings. hash tables lose the ordering property of BSTs, this would reduce lookup overhead when ordering is not required

- In summary, the current design prioritizes ordered traversal over performance. Introducing hash tables in future could yield substantial speed progresses for this dataset, without affecting the correctness of the overall aggregation logic.
- A hash table computes a unique hash value for each key to determine its index.

The main trial with this approach is **hash collisions**, where two different keys produce the same hash index and must share the same storage location. In such cases, the time complexity can degrade toward **$O(n)$** if many items cluster in the same bucket.

Data Structure	Average Case	Worst Case	Maintains Order?	Typical Use
BST / Map (tree-based)	$O(\log n)$	$O(n)$	Yes	Sorted traversal, range queries
Hash Table	$O(1)$	$O(n)$	No	Fast lookups, uniqueness checks
Vector (current duplicate check)	$O(n)$	$O(n)$	No	Sequential iteration only